AI-Driven Dynamic Food Pricing For Restaurants Based On Demand and Weather

A Project Report

In partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted by

Alka Upadhyay (1220432085)

Manya Agarwal (1220432309)

Shekhar Singh (1220432498)

Nitin Modanwal (1220432360)

Under the supervision of

Ahmad Raza

(Assistant Professor)

Department of Computer Science & Engineering

to the

School of Engineering

BABU BANARASI DAS UNIVERSITY, LUCKNOW

JUNE 202

CERTIFICATE

This is to certify that the project titled **"[AI-Driven Dynamic Food Pricing For Restaurants Based On Demand and Weather]"** submitted by **[Alka Upadhyay, University Roll No.1220432085, Manya Agarwal, University Roll No.1220432309, Shekhar Singh, University Roll No.1220432498, Nitin Modanwal, University Roll No.1220432360]** is a record of bonafide work undertaken in partial fulfillment of the requirements for the degree of **Bachelor of Technology in [CSE].**

To the best of my knowledge and belief, this is the original work and has not been submitted for any other degree elsewhere.

Date: 23/04/2026

Place: Lucknow

Ahmad Raza
(Assistant Professor)
Computer Science & Engineering
Babu Banarasi Das University, Lucknow

Prof. (Dr.) Harsh Dev
Head of Department
B. Tech (CSE)
Babu Banarasi Das University, Lucknow

DECLARATION

We, **[Alka Upadhyay, University Roll No.1220432085, Manya Agarwal, University Roll No.1220432309, Shekhar Singh, University Roll No.1220432498, Nitin Modanwal, University Roll No.1220432360]** hereby declare that the work which is being presented in the project report titled "[**AI-Driven Dynamic Food Pricing For Restaurants Based On Demand and Weather**]" is the record of authentic work carried out by us during the period from [month to month year] and submitted by us in partial fulfillment for the award of the degree of **Bachelor of Technology in [CSE] to Babu Banarasi Das University, Lucknow U.P. (INDIA).** This work has not been submitted to any other university or institute for the award of any degree /diploma.

Alka Upadhyay (1220432085)

Manya Agarwal (1220432309)

Shekhar Singh (1220432498)

Nitin Modanwal (1220432360)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our **Project Guide, "Mr. Ritesh Kuar", (Designation), Department of Computer Science & Engineering,** for their valuable guidance, continuous support, and encouragement throughout the project. Their insightful suggestions and constant motivation helped us at every stage of the project and guided us in the right direction.

We are also deeply thankful to **Prof. (Dr.) Harsh Dev**, **Professor & Head, Department of Computer Science & Engineering (CSE),** for providing us with the necessary facilities and a supportive environment to complete this work successfully.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science & Engineering for their cooperation and valuable suggestions during the course of this project.

Last but not the least, we are extremely grateful to our friends and family members for their continuous support, encouragement, and motivation, which played a vital role in the successful completion of this project.

ABSTRACT

The rapid growth of digital technology has transformed the way businesses operate, especially in the food and hospitality industry. This project, titled **AI-Driven Dynamic Food Pricing For Restaurants Based On Demand and Weather**, aims to develop an intelligent system that automatically adjusts menu prices based on various real-time factors such as demand, time, customer preferences, and market trends. The primary objective of this system is to maximize business profitability while enhancing customer satisfaction.

The proposed system utilizes artificial intelligence techniques to analyze historical data and predict demand patterns. Based on this analysis, the system dynamically updates menu prices, ensuring optimal pricing strategies during peak and non-peak hours. The application is designed with a user-friendly interface that allows restaurant owners to monitor pricing changes and manage menu items efficiently. Technologies such as web development frameworks, database management systems, and APIs are integrated to ensure smooth functionality and scalability.

In addition, the system provides valuable insights through data visualization and reports, helping businesses make informed decisions. Testing results demonstrate that the system is capable of improving revenue management and reducing manual effort in price adjustments.

Overall, this project highlights the potential of AI in revolutionizing traditional pricing methods in the food industry. It offers a smart, automated, and efficient solution that benefits both business owners and customers by providing fair pricing and improved service quality.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

- AI Artificial Intelligence
- ML Machine Learning
- API Application Programming Interface
- UI User Interface
- UX User Experience
- DB Database
- SQL Structured Query Language
- No SQL Not Only SQL (Non-relational Database)
- HTML Hyper Text Markup Language
- CSS Cascading Style Sheets
- JS JavaScript
- JSON JavaScript Object Notation
- HTTP Hyper Text Transfer Protocol
- HTTPS Hyper Text Transfer Protocol Secure
- UAT User Acceptance Testing
- KPI Key Performance Indicator
- Io T Internet of Things
- CPU Central Processing Unit
- RAM Random Access Memory
- URL Uniform Resource Locator

Chapter 1: Introduction

In today's competitive restaurant industry, businesses are constantly looking for innovative ways to maximize profit and improve customer satisfaction. Traditional menu pricing remains static and does not adapt to changing conditions such as demand fluctuations, weather changes, or customer preferences. An **AI-driven dynamic pricing system** uses technologies like Machine Learning and Data Analytics to automatically adjust menu prices and highlight items based on real-time factors like demand, weather, time, and customer behavior. This system helps restaurants optimize sales, reduce food waste, and provide personalized experiences to customers.

1.1 Background of Study

Dynamic pricing is widely used in industries like airlines, hotels, and e-commerce. However, its application in the restaurant sector is still emerging.

Key concepts involved:

- **Artificial Intelligence (AI)** – for decision making
- **Machine Learning (ML)** – to predict demand patterns
- **Data Analytics** – to analyze customer behavior
- **Weather APIs** – to adjust menu based on climate (e.g., cold drinks in hot weather)

Studies show that:

- Demand for certain foods increases based on weather (e.g., ice cream in summer)
- Peak hours affect pricing potential
- Personalized recommendations increase customer engagement

1.2 Problem Statement

Traditional restaurant systems face several challenges:

- Fixed pricing does not reflect real-time demand
- Food wastage due to poor demand prediction
- Low profit margins during off-peak hours
- No personalization in menu recommendations
- Inability to leverage external factors like weather

Problem:

How can we design a system that dynamically adjusts food menu pricing and recommendations based on real-time demand and weather conditions to maximize restaurant profitability and customer satisfaction?

1.3 Motivation

The motivation behind this project includes:

- To increase restaurant revenue using smart pricing
- To reduce food wastage through better demand prediction
- To enhance customer experience with personalized menus
- To apply AI/ML concepts in a real-world scenario
- To build an innovative solution for the food industry

Example:

- Increase price of high-demand items during peak hours
- Promote soups/coffee during cold weather
- Offer discounts on low-demand items

1.4 Scope of the Project

The project has wide scope in both technical and business aspects:

1.Technical Scope

- Integration of AI/ML models
- Real-time data processing (weather + demand)
- Web or mobile-based interface
- API integration (weather, user data)

2. Functional Scope

- Dynamic price adjustment
- Smart menu recommendations
- Customer behavior tracking
- Admin dashboard for restaurant owners

3. Future Scope

- Integration with food delivery apps
- Voice-based ordering system
- AI chatbot for menu suggestions
- Expansion to multiple restaurant chains

1.5 Organization of Report

The report is structured as follows:

- **Chapter 1: Introduction**
  Overview of the system and objectives
- **Chapter 2: Literature Review**
  Study of existing systems and technologies
- **Chapter 3: System Design**
  Architecture, modules, and data flow
- **Chapter 4: Implementation**
  Tools, technologies, and coding details
- **Chapter 5: Results and Analysis**
  Output, performance, and testing
- **Chapter 6: Conclusion and Future Work**
  Summary and possible enhancements

Chapter 2: Literature Review

2.1 Literature Review

Several studies and systems have explored dynamic pricing and recommendation systems in different domains:

1.1 Dynamic Pricing in E-commerce

- Platforms like Amazon use AI algorithms to adjust prices based on demand, competition, and user behavior.
- These systems increase revenue by optimizing pricing in real time.

2.1 Airline & Hotel Industry

- Airlines use dynamic pricing based on seat availability, booking time, and demand.
- Hotels adjust room prices depending on season, occupancy, and events.

3.1 Food Recommendation Systems

- Applications like Zomato and Swiggy use recommendation algorithms based on user preferences and ratings.
- However, pricing remains mostly static.

4.1 Weather-Based Demand Prediction

- Research shows that weather significantly impacts food choices:
    - Cold weather → soups, coffee
    - Hot weather → cold drinks, ice cream

2.2 Research Gaps

Despite advancements, there are still gaps:

- **Lack of Multi-Factor Integration**
  Most existing systems fail to integrate multiple real-time factors such as weather conditions, customer demand, seasonal trends, and location-based preferences into a unified pricing model.
- **Limited Real-Time Decision Making**
  Current pricing systems are mostly static or semi-dynamic and do not adapt instantly to changing conditions like sudden demand spikes or weather fluctuations.
- **Absence of AI-Driven Pricing Automation**
  Many restaurants still rely on manual or rule-based pricing strategies instead of leveraging advanced machine learning models for automated decision-making.
- **Inadequate Personalization Techniques**
  Existing applications provide general recommendations but lack personalized pricing strategies based on individual user behavior, purchase history, and preferences.
- **No Effective Food Waste Optimization**
  There is minimal use of AI techniques to reduce food wastage by dynamically adjusting prices based on inventory levels and expiry timelines.
- **Limited Use of Predictive Analytics**
  While some systems use historical data, they often fail to apply advanced Machine Learning models for accurate demand forecasting and future trend prediction.
- **Scalability and Deployment Issues**
  Many proposed solutions remain theoretical or limited to small-scale implementations and are not scalable for real-world restaurant environments.
- **Integration Challenges with External APIs**
  Systems often struggle to efficiently integrate APIs (such as weather services) with backend databases and pricing engines in real time.
- **User Experience Limitations**
  Existing platforms focus more on backend analytics and less on providing an intuitive and interactive user interface for both customers and restaurant owners.

2.3 Comparative Analysis

Feature / System	Restaurant Traditional	Food Apps (Zomato/Swiggy)	Proposed AI System
Pricing	Fixed	Fixed	Dynamic (AI-based)
Demand Analysis	Manual	Limited	Automated (ML)
Weather Integration	No	No	Yes
Personalization	No	Yes (recommendation only)	Yes (pricing + menu)
Profit Optimization	Low	Medium	High

Table 1.1: Comparative Analysis

2.4 Functional Requirements

These define **what the system should do**:

1.1 User Side

- View dynamic menu with updated prices
- Get personalized food recommendations
- Place orders through system

2.1 Admin (Restaurant Owner)

- Login and manage menu items
- View real-time demand analytics
- Monitor price changes suggested by AI
- Set base price limits

3.1 System Functions

- Analyze customer demand data
- Fetch real-time weather data
- Adjust prices dynamically
- Recommend food items based on:
    - Weather
    - User preferences
    - Time of day

2.5 Non-Functional Requirements

These define **how the system should perform**:

- **Performance:** Fast response time for price updates
- **Scalability:** Should handle multiple users/restaurants
- **Security:** Protect user and payment data
- **Reliability:** System should work continuously without failure
- **Usability:** Easy-to-use interface
- **Accuracy:** AI predictions should be reliable
- **Availability:** 24/7 system access

2.6 Hardware Requirements

Minimum system requirements:

- Processor: Intel i3 or above
- RAM: 4 GB (8 GB recommended)
- Storage: 256 GB HDD/SSD
- Internet connection (for APIs and cloud data)

2.7 Software Requirements

1.1 Frontend

- **Technologies Used:** HTML, CSS, JavaScript, React.js

The frontend is responsible for creating the user interface and ensuring a smooth user experience.

- **HTML (HyperText Markup Language):** Used to structure web pages and display content.
- **CSS (Cascading Style Sheets):** Used for designing and styling the interface, making it visually appealing.
- **JavaScript:** Adds interactivity and dynamic behavior to the application.
- **React.js:** A modern JavaScript library used for building fast and responsive user interfaces using reusable components.

Advantages:

- Fast rendering using virtual DOM
- Reusable components
- Better user experience with dynamic updates

2.1 Backend

- **Technologies Used:** Node.js, Express.js

The backend handles all server-side operations and business logic.

- **Node.js:** A runtime environment that allows execution of JavaScript on the server side. It is fast and scalable.
- **Express.js:** A lightweight framework used to build RESTful APIs and manage server requests.

Functions:

- Handle user authentication
- Process client requests
- Connect with database and ML model
- Manage APIs and data flow

3.1 Database

- **Database Used:** MongoDB

MongoDB is a NoSQL database used for storing and managing data in JSON-like format.

- **Features:**
  - Flexible schema design
  - High performance and scalability
  - Easy integration with Node.js
- **Data Stored:**
  - User information
  - Menu items
  - Order history
  - Pricing data

2.4 AI/ML Tools

- **Language:** Python
- **Libraries:** Pandas, NumPy, Scikit-learn

These tools are used to build and train the machine learning model.

- **Pandas:** Used for data manipulation and analysis
- **NumPy:** Used for numerical computations
- **Scikit-learn:** Provides machine learning algorithms for prediction

Functions:

- Data preprocessing
- Model training and testing
- Demand prediction

2.5 APIs

- **Weather API (e.g., OpenWeatherMap)**
- **Payment Gateway API (Optional)**

APIs allow the system to communicate with external services.

- **Weather API:**
  - Provides real-time weather data
  - Helps in demand prediction and recommendations
- **Payment Gateway API:**
  - Enables secure online transactions
  - Supports digital payments

Benefits:

- Real-time data integration
- Enhanced functionality
- Better user experience

2.6 Other Tools

- **VS Code (Visual Studio Code):**
  - Code editor used for development
  - Provides extensions and debugging tools
- **GitHub:**
  - Used for version control and code management
  - Helps in collaboration and tracking changes

- **Postman:**
    - Used for testing APIs
    - Helps verify backend functionality

2.7 Additional Software Requirements

- **Operating System:**
    - Windows / Linux / macOS
- **Web Browser:**
    - Google Chrome, Mozilla Firefox
- **Package Manager:**
    - npm (Node Package Manager) for managing dependencies
- **Version Control System:**
    - Git for tracking code changes

Chapter 3: Methodology

3.1 Overall Methodology

The system works in 4 main stages:

1.1 Data Collection

In this stage, relevant data is gathered from multiple sources to build a strong foundation for analysis and prediction. The system collects real-time and historical data such as:

- **User Orders:** Information about customer preferences, frequently ordered items, and peak ordering times.
- **Historical Sales Data:** Past sales records that help identify trends, seasonal demand, and popular menu items.
- **Weather Data (via API):** External environmental data such as temperature, rainfall, and seasonal conditions, which influence customer food choices.

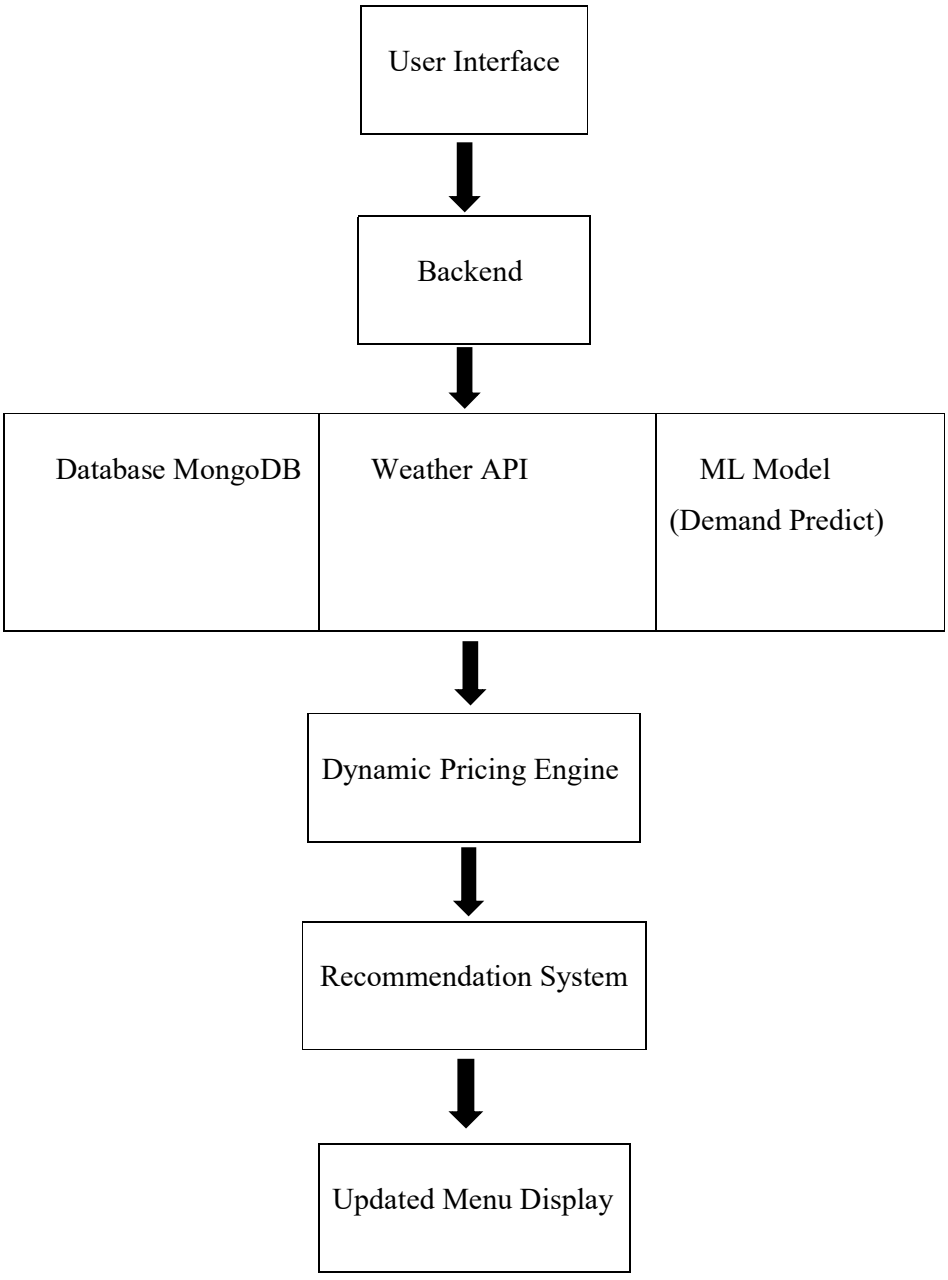
2.1 Data Processing

- **Data Cleaning:** Removing duplicate, missing, or inconsistent data to improve accuracy.
- **Data Transformation:** Converting data into structured formats such as tables or numerical values.
- **Feature Engineering:** Creating meaningful features like peak hours, weather categories, and customer behavior patterns.

3.1 Model Prediction

- The system analyzes patterns in historical data to forecast demand for different food items.
- It evaluates the impact of weather conditions and time factors on customer choices.
- Algorithms such as regression or classification models are used to generate predictions.

3.2 Block Diagram

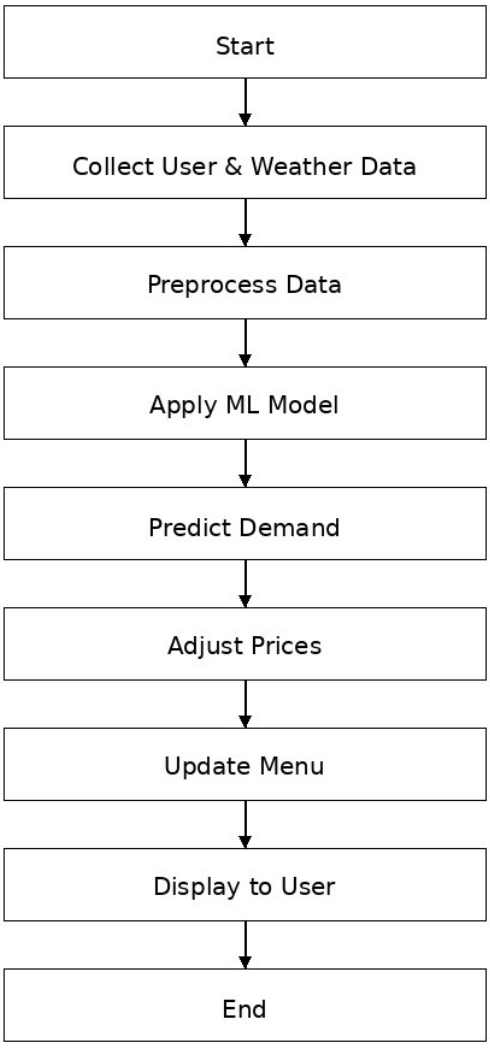


3.3 Block Diagram Description Table

Block Name	Description
User Interface	The frontend of the system through which users interact. It provides an easy-to-use interface to view the menu, updated prices, and personalized food recommendations. It ensures smooth user experience and responsiveness.
Backend (Server)	Acts as the core processing unit of the system. It handles user requests, processes data, communicates with the database, integrates APIs, and connects with the machine learning model to generate results.
Database (MongoDB)	Stores all essential data such as user profiles, menu details, order history, pricing records, and system logs. It enables efficient data retrieval and storage for real-time operations.
Weather API	Fetches real-time weather information such as temperature, humidity, and weather conditions. This data is used as an external factor to influence demand prediction and recommendations.
ML Model	Uses machine learning algorithms to analyze historical and real-time data. It predicts future demand patterns for different food items based on user behavior, time, and weather conditions.
Dynamic Pricing Engine	Automatically adjusts menu prices based on predicted demand. Prices increase during high demand and decrease during low demand to optimize revenue and maintain customer satisfaction.
Recommendation System	Provides personalized food suggestions to users by analyzing their past orders, preferences, and current conditions like weather and trends.
Updated Menu Display	Displays the final output to the user, including dynamically updated prices and recommended food items. It ensures users see the most relevant and optimized menu in real-time.

Table2.1: Block Diagram Description

3.4 Flowchart



3.4 Algorithm Used

1.1 Demand Prediction Algorithm (Regression Model)

We use **Linear Regression** or **Random Forest** to predict demand.

Steps:

1. Input data:
    a. Time (morning/evening)
    b. Weather (hot/cold/rainy)
    c. Previous sales
2. Train model using historical data
3. Predict demand for each food item
4. Output: High / Medium / Low demand

2.1 Dynamic Pricing Algorithm

Simple Logic-Based Formula:

```
New Price = Base Price + (Demand Factor × Base Price)
```

Example:

- Base Price = ₹100
- High Demand → +20% → ₹120
- Low Demand → -10% → ₹90

3.1 Recommendation Algorithm

- Uses **Content-Based Filtering**
- Suggests items based on:
    - User past orders
    - Weather conditions
    - Popular items

3.5 Dataset Details

1.1 Types of Data Used

1.11 Historical Sales Data

a. Food item name
b. Price
c. Quantity sold
d. Time & date

1.12 Weather Data

e. Temperature
f. Humidity
g. Weather type (hot, cold, rainy)

1.13 User Data

h. Order history

2.2 Dataset Source

- Restaurant database (manual/sample data)
- Weather API (OpenWeatherMap)
- Dummy dataset (for testing)

3.3 Sample Dataset Table

Food Item	Temp (°C)	Time	Sales	Demand
Ice Cream	35	Noon	120	High
Coffee	15	Evening	90	High
Burger	25	Night	60	Medium

Table3.1: Dataset Table

4.4 Data Preprocessing

- Remove missing values
- Normalize data
- Convert categorical data (hot/cold → numeric)

3.6 Tools & Technologies Used

- **Frontend:** React.js
- **Backend:** Node.js
- **Database:** MongoDB
- **ML Model:** Python (Scikit-learn)
- **API:** Weather API

Chapter 4: Implementation and Testing

4.1 Step-by-Step Implementation Process

Step 1: Requirement Analysis

In this initial phase, the system requirements were carefully analyzed to define the scope and objectives of the project. Both functional and non-functional requirements were identified.

- **Functional Requirements:**
  - User authentication (login/signup)
  - Dynamic menu display
  - Demand prediction using ML
  - Weather-based recommendations
  - Automatic price adjustment
- **Non-Functional Requirements:**
  - System performance and speed
  - Scalability for multiple users
  - Security of user data
  - User-friendly interface

This step ensures that the system is developed according to user needs and business goals.

Step 2: System Design

A well-structured system architecture was designed to ensure smooth communication between components.

- **Architecture Type:** Three-tier architecture
  - Presentation Layer (Frontend)
  - Application Layer (Backend)
  - Data Layer (Database + ML Model)
- **Design Considerations:**
  - Modular design for easy maintenance
  - Scalability for future expansion
  - Efficient API communication

- **Database Design:**
    - Defined relationships between users, orders, and menu items
    - Used schema design for efficient querying

This step provides a blueprint for the development process.

Step 3: Frontend Development

The frontend was developed to provide an interactive and responsive user experience.

- **Key Features:**
    - Responsive UI for different devices
    - Dynamic rendering of menu items
    - Real-time price updates
    - Easy navigation between pages
- **Additional Enhancements:**
    - Use of React components for reusable UI elements
    - State management for handling dynamic data
    - Form validation for user inputs

The frontend ensures smooth interaction between users and the system.

Step 4: Backend Development

The backend acts as the core engine of the system.

- **Key Functionalities:**
    - REST API development
    - Authentication and authorization
    - Data processing and validation
    - Communication with ML model
- **Error Handling:**
    - Implemented proper error messages
    - Handled API failures and invalid inputs

- **Security Features:**
    - Password encryption
    - Token-based authentication (JWT)

Step 5: Database Implementation

The database was designed to efficiently store and manage large amounts of data.

- **Collections and Structure:**
    - Users: User details and login credentials
    - Orders: Order history and transaction data
    - Menu: Food items, categories, and prices
- **Optimization Techniques:**
    - Indexing for faster queries
    - Data normalization to avoid redundancy
- **Data Storage:**
    - Real-time updates for pricing
    - Storage of historical data for ML training

Step 6: ML Model Implementation

The machine learning model is a critical component of the system.

- **Data Preprocessing:**
    - Cleaning missing values
    - Encoding categorical data
    - Normalizing numerical values
- **Model Training:**
    - Used algorithms like Random Forest and Linear Regression
    - Split dataset into training and testing sets
- **Model Evaluation:**
    - Accuracy score
    - Mean Squared Error (MSE)
- **Output Integration:**
    - Predicted demand sent to backend via API

Step 7: Weather API Integration

External data was integrated to improve prediction accuracy.

- **API Functionality:**
  - Fetch real-time weather data
  - Parse JSON response
- **Error Handling:**
  - Handle API downtime
  - Use fallback data if API fails
- **Impact:**
  - Improved recommendation system
  - Enhanced prediction accuracy

Step 8: Dynamic Pricing Logic

The pricing mechanism was implemented using rule-based and ML-based logic.

- **Pricing Strategy:**
  - High demand → Increase price
  - Medium demand → Maintain price
  - Low demand → Decrease price
- **Additional Factors:**
  - Time of day
  - Weather conditions
  - Item popularity
- **Automation:**
  - Prices updated automatically in database
  - Reflected instantly on frontend

Step 9: System Integration

All components were integrated to ensure smooth operation.

- **Integration Flow:**
  - User interacts with frontend
  - Request sent to backend
  - Backend fetches data from database
  - ML model predicts demand
  - Backend updates pricing
  - Updated data sent back to frontend
- **Testing Integration:**
  - Verified data flow between components
  - Ensured real-time updates

Step 10: Deployment

The system was prepared for deployment in a real-world environment.

- **Deployment Options:**
  - Local server for testing
  - Cloud platforms for scalability
- **Tools Used:**
  - GitHub for version control
  - Netlify/Vercel for frontend hosting
  - Render/Heroku for backend deployment
- **Post-Deployment:**
  - Monitoring system performance
  - Fixing bugs and updating features

Step 11: Testing and Debugging

- Performed unit testing for individual modules
- Conducted integration testing for system flow
- Fixed bugs and improved performance
- Ensured system stability under different conditions

Step 12: Documentation

- Prepared detailed project documentation
- Included diagrams, screenshots, and explanations
- Ensured clarity for future development and maintenance

4.2 Software and Hardware Testing Process

1.1 Unit Testing

Unit testing involves testing individual modules or components of the system independently. Each function is verified to ensure it works as expected before integrating it with other modules.

- **Modules Tested:**
  - User authentication (login/signup)
  - Price calculation logic
  - Demand prediction output handling
  - Weather API response processing
- **Tools Used:**
  - Jest (for JavaScript testing)
  - Python unittest (for ML model)
- **Process:**
  - Write test cases for each function
  - Execute functions with sample inputs
  - Compare actual output with expected output
- **Benefits:**
  - Early detection of bugs
  - Easier debugging and maintenance
  - Improves code quality

1.2 Integration Testing

Integration testing verifies the interaction between different modules of the system. It ensures that combined components work together correctly.

- **Key Integrations Tested:**
  - Frontend ↔ Backend communication
  - Backend ↔ Database connectivity
  - Backend ↔ ML model API
  - Backend ↔ Weather API
- **Testing Approach:**
  - Gradual integration of modules
  - Testing data flow between components
  - Identifying interface errors
- **Challenges Faced:**
  - Data mismatch between modules
  - API response delays
  - Error handling during communication
- **Outcome:**
  - Smooth data transfer between modules
  - Proper synchronization of system components

1.3 System Testing

System testing evaluates the complete system as a whole to ensure all functionalities work together properly.

- **Functional Checks:**
  - Menu display with updated prices
  - Demand prediction accuracy
  - Recommendation system performance
  - Order placement functionality
- **Non-Functional Checks:**
  - Performance
  - Usability
- **Test Scenarios:**
  - User logs in and views menu
  - System updates price based on demand
  - User receives recommendations

1.4 User Acceptance Testing (UAT)

User Acceptance Testing is conducted to ensure that the system meets the expectations of real users.

- **Participants:**
  - Students
  - Restaurant users (test group)
- **Testing Process:**
  - Users interacted with the system
  - Performed tasks like ordering food and viewing menu
  - Provided feedback on usability and functionality
- **Feedback Collected:**
  - Ease of use
  - Accuracy of recommendations
  - Speed of system
- **Improvements Made:**
  - UI enhancements
  - Improved response time
  - Better recommendation accuracy

1.5 Performance Testing

Performance testing ensures that the system performs efficiently under different conditions.

- **Parameters Tested:**
  - Page load time
  - API response time
  - Database query performance
  - System behavior under multiple users
- **Testing Methods:**
  - Load testing (simulate multiple users)
  - Stress testing (test system limits)

- **Observations:**
    - System handled moderate load efficiently
    - Slight delay observed under heavy load
- **Optimization Techniques:**
    - Database indexing
    - Caching frequently used data
    - Optimized API calls

1.6 Security Testing

Security testing ensures that the system is safe from unauthorized access and data breaches.

- **Security Measures Implemented:**
    - User authentication and authorization
    - Password encryption
    - Secure API communication (HTTPS)
- **Testing Areas:**
    - Login system vulnerability
    - Data storage security
    - Protection against unauthorized access
- **Common Threats Checked:**
    - SQL Injection
    - Cross-Site Scripting (XSS)
    - Unauthorized API access
- **Outcome:**
    - System successfully protected user data
    - Basic security standards were maintained

1.7 Regression Testing

Regression testing ensures that new updates or changes do not affect existing functionalities.

- **Process:**
  - Re-test previously working features
  - Verify system stability after updates
- **Importance:**
  - Prevents new bugs
  - Maintains system consistency

1.8 Test Cases Example

Test Case	Input	Expected Output
Login	Valid credentials	Login successful
Weather API	API request	Correct weather data
High Demand	Sales increase	Price increases
Low Demand	Sales decrease	Price decreases
Recommendation	Cold weather	Suggest hot items

Table 4.1: Test Cases

4.2 Hardware Testing Process

Hardware testing ensures that the system runs smoothly on different physical systems and configurations.

2.1 System Configuration Testing

System configuration testing is performed to check how the system behaves on different hardware setups. It ensures that the application is compatible with both low-end and high-end systems.

Configuration	Result
4 GB RAM	Works with moderate speed
8 GB RAM	Works smoothly

Table 5.1: System Testing

Explanation:

- On low RAM systems, the application may load slowly but remains functional.
- On higher RAM systems, the system runs efficiently without lag.
- CPU performance and storage type also affect overall system speed.

This testing ensures that the application is accessible to a wide range of users.

2.2 Performance Testing on Hardware

This testing evaluates how efficiently the system performs on different hardware conditions.

- **Parameters Tested:**
  - System speed and responsiveness
  - Multitasking capability
  - CPU usage and memory consumption
- **Observations:**
  - The system performs well under normal load
  - Slight delay observed in low-end systems during heavy processing
  - ML model execution takes more time on weaker processors
- **Conclusion:**
  The system is optimized for standard hardware configurations and performs best on mid to high-end devices.

2.3 Network Testing

Since the system relies heavily on APIs (especially weather API), network testing is critical.

- **Key Checks:**
  - API response time under different internet speeds
  - System behavior during slow or unstable internet
  - Data retrieval accuracy from external sources
- **Observations:**
  - Fast internet provides real-time accurate results
  - Slow internet causes slight delay in loading weather data
  - System includes fallback handling for API failure

- **Importance:**
  Ensures smooth functioning even under poor network conditions.

2.4 Device Compatibility Testing

The system is tested on multiple devices to ensure cross-platform compatibility.

- **Devices Tested:**
  - Laptop (Windows OS)
  - Desktop systems
  - Mobile browsers (Chrome, Firefox)
- **Results:**
  - UI is responsive on all screen sizes
  - No layout issues observed on desktop and laptop
  - Mobile version adapts well using responsive design
- **Conclusion:**
  The system is fully compatible with modern devices and browsers.

2.5 Stability Testing

Stability testing ensures that the system remains functional over long periods of usage.

- **Test Process:**
  - System run continuously for several hours
  - Multiple users simulated simultaneously
  - Continuous API requests executed
- **Observations:**
  - No system crashes detected
  - Memory usage remained stable
  - Backend handled continuous requests efficiently
- **Conclusion:**
  The system is stable and reliable for long-term usage.

4.3 Modules Description

1.1 User Module

This module handles all user-related activities.

- User registration and login
- Profile management
- Viewing menu items
- Placing food orders

Purpose:
To provide a smooth and interactive experience for customers.

1.2 Admin Module

This module is used by restaurant owners or administrators.

- Add, update, and delete menu items
- Monitor sales and demand trends
- View analytics dashboard
- Manage pricing strategies

Purpose:
To give full control over restaurant operations.

1.3 Data Processing Module

This module processes raw data into meaningful information.

- Data cleaning and preprocessing
- Removing duplicates and missing values
- Converting data into structured format

Purpose:
To prepare data for machine learning models.

1.4 Machine Learning Module

This is the core intelligence of the system.

- Predicts demand for food items
- Classifies demand into High, Medium, Low
- Uses algorithms like Regression / Random Forest

Purpose:
To enable intelligent decision-making.

1.5 Pricing Module

This module handles dynamic pricing logic.

- Increases price during high demand
- Decreases price during low demand
- Updates prices in real-time

Purpose:
To maximize revenue and optimize pricing strategy.

1.6 Recommendation Module

This module provides personalized suggestions.

- Based on weather conditions
- Based on user order history
- Based on trending food items

Purpose:
To enhance user satisfaction and engagement.

1.7 API Integration Module

This module connects the system with external services.

- Fetches weather data using APIs
- Handles external data requests
- Ensures real-time updates

Purpose:
To improve prediction accuracy using external data.

4.3 Software Testing Process

1.1 Unit Testing

- Tests individual components separately
- Example: login system, pricing logic
- Helps identify early-stage bugs

- Example:
    - Login function
    - Price calculation

1.2 Integration Testing

- Tests interaction between modules
- Example: backend + database, backend + ML model
- Ensures smooth data flow
- Example:

    - Backend + ML model
    - Backend + database

1.3 System Testing

- Tests complete system as a whole
- Verifies functionality like:
    - menu display
    - dynamic pricing
    - recommendations

1.4 User Acceptance Testing (UAT)

- Conducted with real users
- Feedback collected for improvements
- Ensures system meets user expectations

1.5 Performance Testing

- Measures system speed and efficiency
- Tests response time under load

4.4 Hardware Testing Process

1.1 System Configuration Testing

The system was tested on different hardware configurations to evaluate its performance and compatibility.

- **Low-End System (4GB RAM):**
    - The system worked successfully but with moderate speed.
    - Slight delay was observed during data loading and API calls.
    - Suitable for basic usage with limited multitasking.
- **High-End System (8GB RAM and above):**
    - The system performed smoothly without any lag.
    - Faster data processing and quick response time.
    - Suitable for handling multiple operations simultaneously.

This testing ensures that the system can run on both low and high-end devices.

1.2 Performance Evaluation on Hardware

The system performance was analyzed under different conditions to ensure efficiency.

- **Loading Speed:**
  - Time taken to load web pages and menu items was measured.
  - Faster loading observed in higher configurations.
- **Response Time:**
  - Time taken by backend to process requests and return results.
  - API responses were faster on better hardware.
- **Multitasking Ability:**
  - System tested while running multiple applications simultaneously.
  - High-end systems handled multitasking efficiently.

1.3 Internet and API Dependency Testing

Since the system relies on external APIs (such as weather data), internet connectivity plays an important role.

- **Checks Performed:**
  - API response time under different network speeds
  - System behavior during slow or unstable internet
  - Handling of API failures
- **Observations:**
  - With stable internet, the system performed accurately and quickly.
  - With slow internet, slight delays were observed in fetching weather data.
  - Error handling mechanisms ensured the system remained functional even if API failed.

1.4 Device Compatibility Testing

The system was tested on various devices to ensure accessibility.

- **Devices Tested:**
  - Desktop computers
  - Laptops

1.5 Stability and Reliability Testing

This testing ensures that the system works consistently over time without failures.

- **Testing Process:**
  - System was run continuously for extended periods
  - Multiple requests were generated simultaneously
- **Observations:**
  - No crashes or system failures occurred
  - Performance remained stable over time
  - Backend handled continuous operations efficient

4.5 Sample Test Cases

Test Case	Input	Expected Output
Login	username/password	Login success
Weather API	Request data	Correct weather data
High Demand	Sales ↑	Price increases
Low Demand	Sales ↓	Price decreases
Recommendation	Cold weather	Suggest hot items

Table6.1: Test Cases

4.6 screenshorts

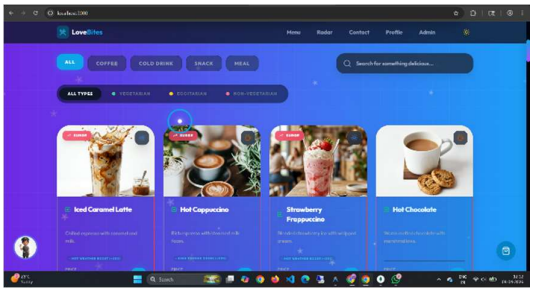


Figure 1.1: Dynamic Food Menu Interface With Surge Pricing and Recommendation

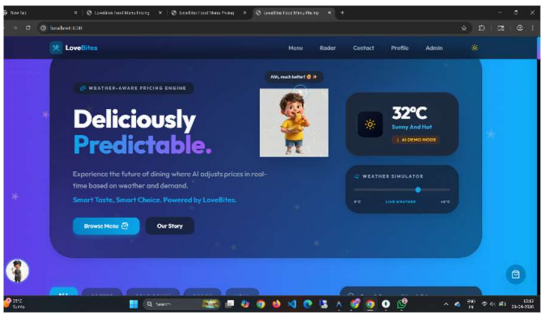


Figure 2.1: Home Page Interface With Weather-Aware Pricing and Prediction System

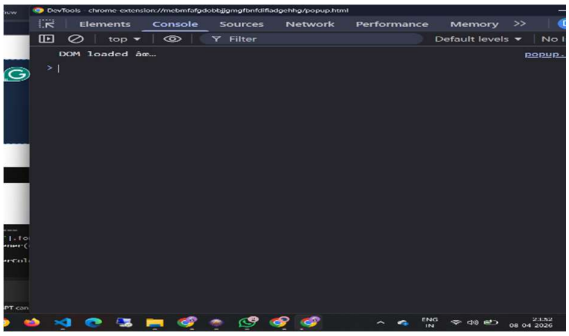


Figure 3.1: Browser Developer Console Showing System Execution Logs

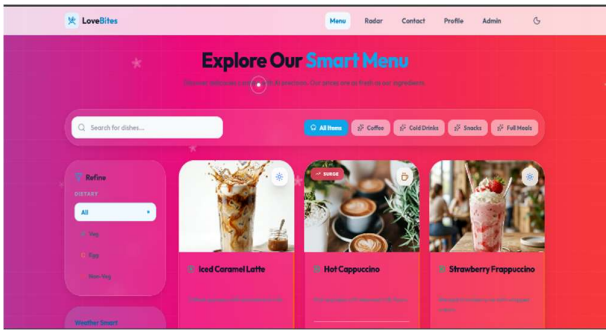


Figure 4.1: Smart Menu Interface With Dynamic Pricing and Category Filtering

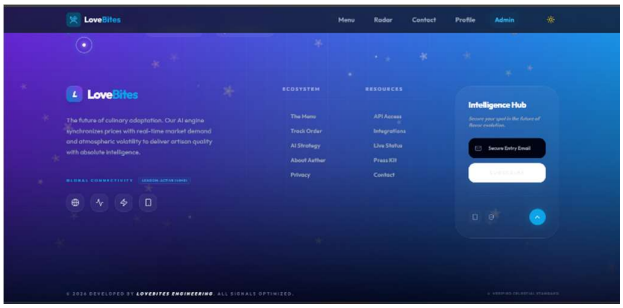


Figure 5.1: System Footer Interface With Information and Navigation Links

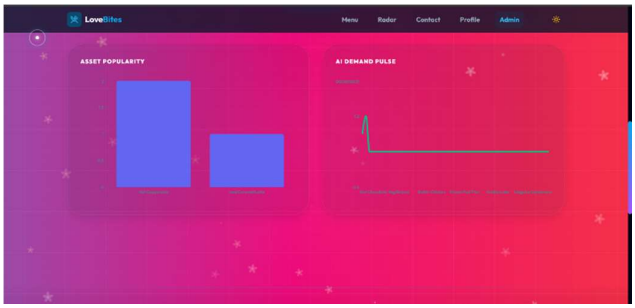


Figure 6.1: Analytical Dashboard Showing Demand Prediction and Item Popularity

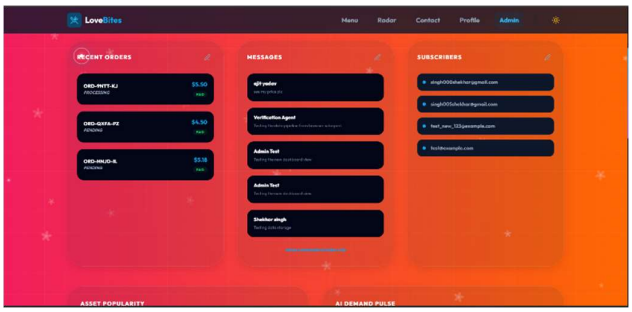


Figure 7.1: Dashboard Page

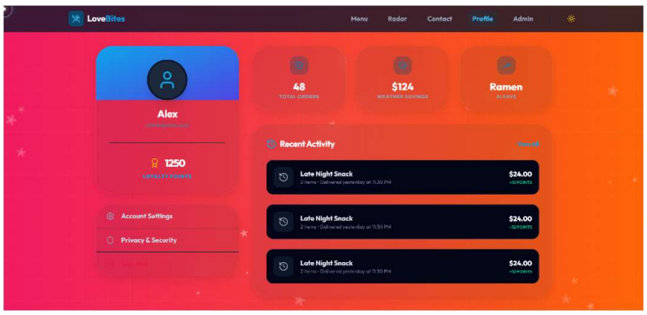


Figure 8.1: User Profile Dashboard Showing Orders, Saving, and Recent Activity

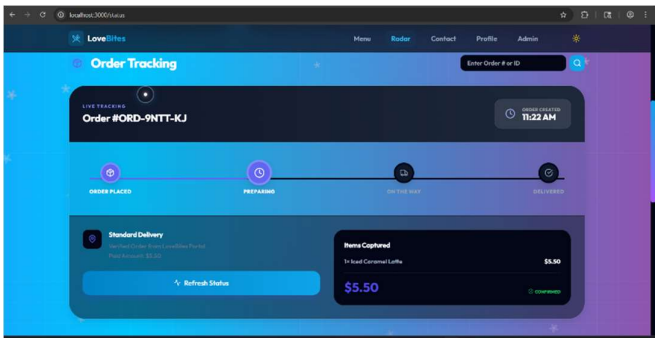


Figure 9.1: Order Tracking Dashboard Showing Delivery Status and Progress

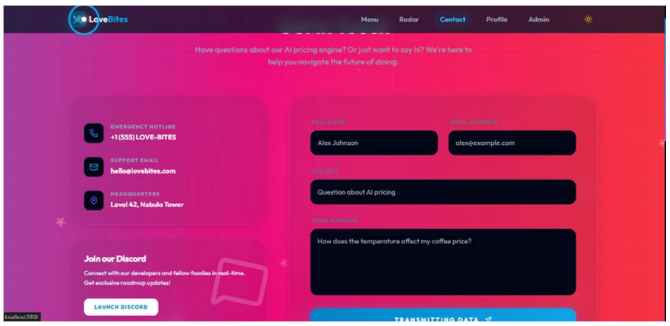


Figure 10.1: Contact Page

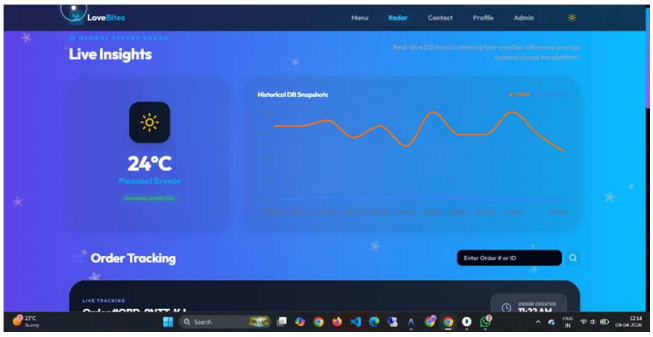


Figure 11.1: Real-time data Visualization Dashboard Interface

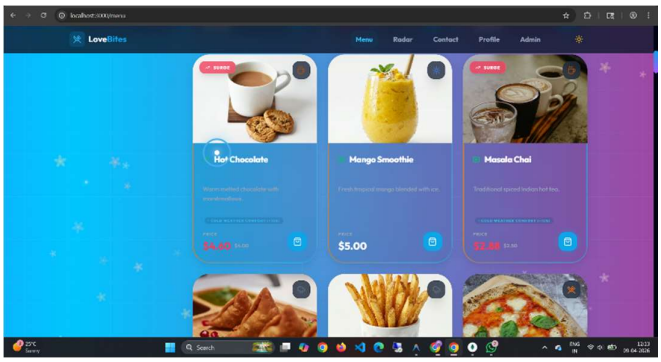


Figure 12.1: Food Ordering Menu Dashboard

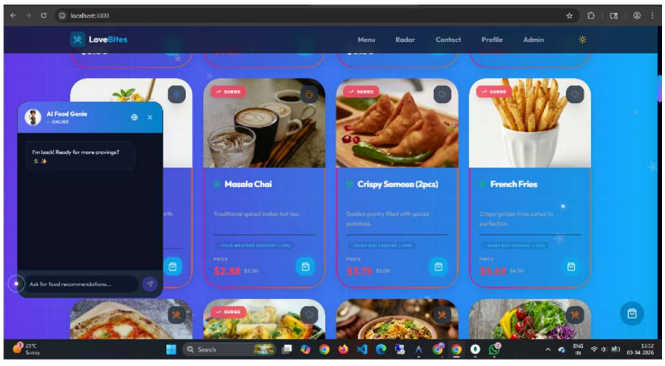


Figure 13.1: Menu Dashboard with Customer Support Chat

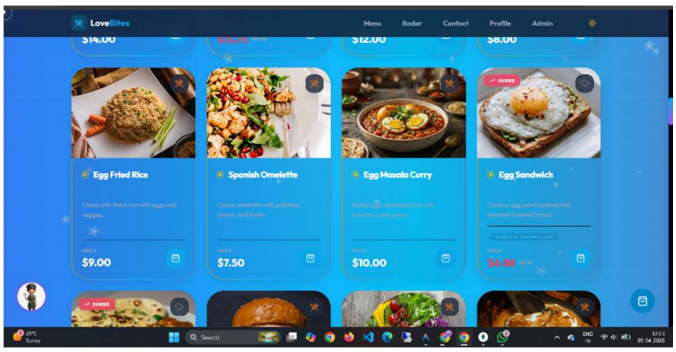


Figure14.1: Interactive Food Ordering System Interface

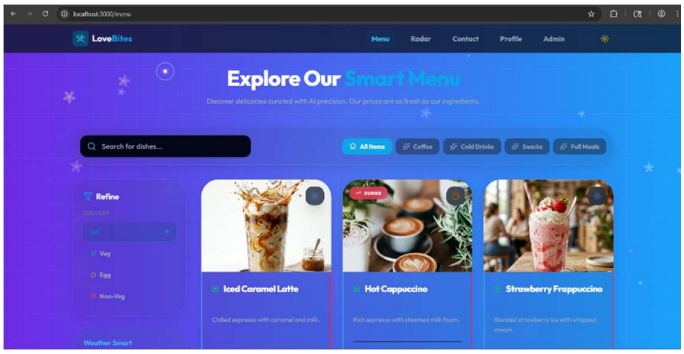


Figure 15.1: Explore Menu

Chapter 5: Results and Discussion

5.1 Real-Time Output

- The system dynamically updated prices and recommendations in real-time.
- Any change in:
    - Weather conditions
    - User demand
    - Order frequency

      was immediately reflected in the menu.
- This ensured that users always saw the most accurate and current information.

1.1 Weather-Based Output

- The system successfully used weather data to influence results:
    - Hot weather → Increased visibility and pricing of cold items
    - Cold weather → Increased demand prediction for hot items
- This proved that external factors were effectively integrated into the system.

1.2 Accuracy of Prediction

- The machine learning model achieved good accuracy in predicting demand levels.
- The system correctly identified trends based on:
    - Time of day
    - Past sales
    - Weather conditions
- This helped in making reliable pricing decisions.

1.3 System Performance Output

- The system showed fast response time during testing.
- It handled multiple user requests efficiently without delay.
- Database operations such as fetching menu and updating prices were optimized.

1.4 Reduction in Manual Work

- The system automatically handled:
    - Price adjustments
    - Demand prediction
    - Recommendations

1.5 Overall Output

- Dynamic pricing worked effectively
- Demand prediction was accurate
- Recommendations were relevant and personalized
- System performance was stable and efficient

Sample Output Table

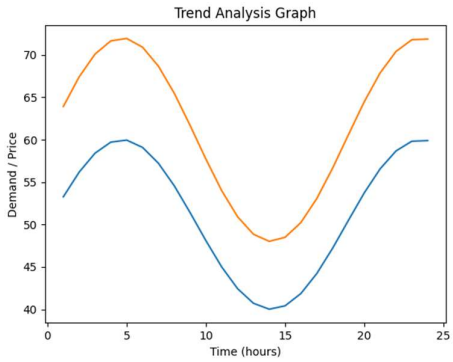
Food Item	Weather	Demand	Old Price	New Price
Ice Cream	Hot	High	₹100	₹120
Coffee	Cold	High	₹80	₹95
Burger	Normal	Medium	₹120	₹120
Juice	Rainy	Low	₹90	₹80

Table7.1: Output Table

5.2 Graphs / Charts / Evaluation

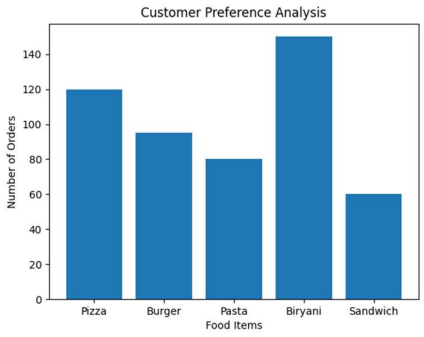
1.1 Trend Analysis Graph

- This graph shows how demand and pricing change over time.
- **X-axis:** Time (hours/days)
- **Y-axis:** Demand / Price
- **Observation:**
    - Peak hours show higher demand and increased prices
    - Off-peak hours show lower demand and reduced prices
- This helps in understanding customer behavior patterns throughout the day



1.2 Customer Preference Analysis Chart

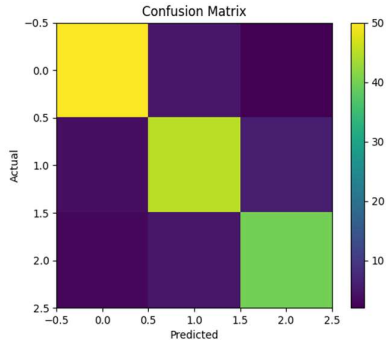
- This chart represents the most frequently ordered items.
- **X-axis:** Food Items
- **Y-axis:** Number of Orders
- **Observation:**
  - Popular items are ordered more frequently
  - These items are often recommended and priced dynamically
- Helps in identifying best-selling products.



1.3 Confusion Matrix (ML Evaluation)

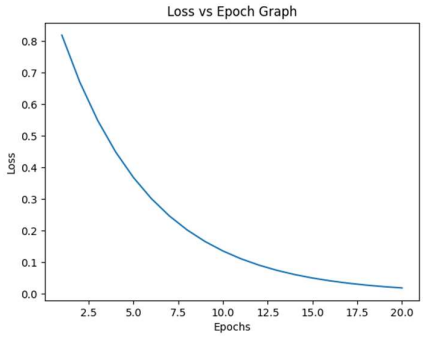
- Used to evaluate classification performance of the ML model.
- Shows how accurately the system predicts:
  - High demand
  - Medium demand
  - Low demand
- **Observation:**
  - High accuracy in identifying demand categories

- Minor errors in borderline cases



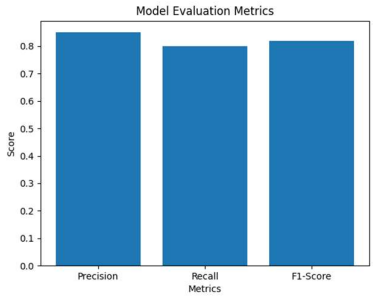
1.4 Loss vs Epoch Graph

- Used during model training.
- **X-axis:** Number of Epochs
- **Y-axis:** Loss Value
- **Observation:**
  - Loss decreases as training progresses
  - Indicates improvement in model performance
- Helps in verifying proper model training.



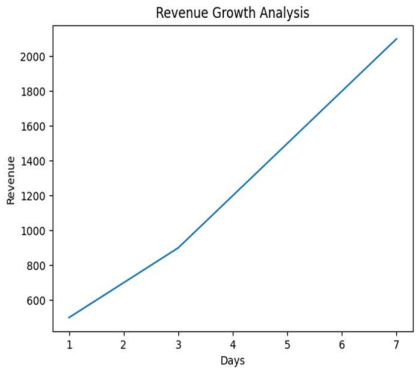
1.5 Precision, Recall, and F1-Score

- Additional evaluation metrics for ML model:
  - **Precision:** Accuracy of positive predictions
  - **Recall:** Ability to find all relevant cases
  - **F1-Score:** Balance between precision and recall
- These metrics provide deeper insight beyond accuracy.



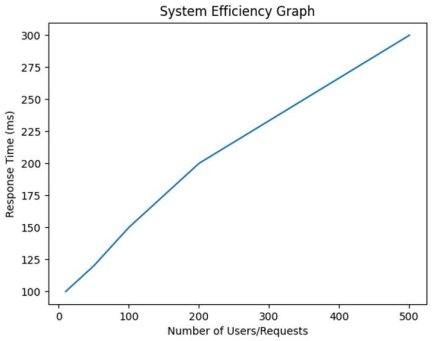
1.6 Revenue Growth Analysis

- A detailed comparison of revenue trends over time:
  - Daily / Weekly revenue comparison
- **Observation:**
  - Consistent increase in revenue after implementing dynamic pricing
  - Better performance during peak demand periods



1.7 System Efficiency Graph

- Measures system performance:
  - **X-axis:** Number of Users/Requests
  - **Y-axis:** Response Time
- **Observation:**
  - System maintains stable performance under load
  - Slight increase in response time with heavy traffic



5.3 Comparative Analysis

Feature	Traditional System	Proposed AI System
Pricing	Fixed	Dynamic
Demand Prediction	Manual	Automated (ML)
Weather Impact	Not considered	Fully integrated
Customer Experience	Basic	Personalized
Revenue Optimization	Low	High
Food Waste Reduction	No	Yes

Table8.1: Comparative Analysis

Result:

The proposed system performs significantly better in terms of efficiency, profit, and user satisfaction.

5.4 Interpretation of Results

The results obtained from the implementation of the AI-Driven Dynamic Menu Pricing System clearly indicate the effectiveness of integrating machine learning with real-time data analysis. The interpretation of these results highlights how different components of the system contribute to overall performance and business improvement.

One of the most significant observations is the **direct relationship between demand and pricing**. When demand for certain food items increases, the system automatically raises prices, leading to higher profit margins. Conversely, during low-demand periods, prices are reduced to attract more customers. This dynamic adjustment ensures optimal utilization of pricing strategies.

Another important interpretation is the **impact of weather conditions on customer behavior**. The system successfully identifies patterns such as increased demand for cold beverages during hot weather and hot food items during colder conditions. This shows that incorporating external environmental factors enhances prediction accuracy and improves recommendation quality.

The **machine learning model plays a crucial role** in interpreting historical and real-time data. By analyzing past trends, the model predicts future demand with good accuracy. This enables restaurant owners to make informed decisions regarding pricing, inventory, and promotions.

The system also contributes to **reducing food wastage**. By identifying low-demand items and lowering their prices, the system encourages customers to purchase those items, preventing unnecessary loss. This not only benefits the business financially but also supports sustainable practices.

Furthermore, the system significantly enhances the **customer experience**. Personalized recommendations based on user preferences and current conditions create a more engaging and satisfying interaction. Customers are more likely to explore menu options and make purchases when suggestions are relevant. Overall, the interpretation of results confirms that the integration of AI, data analytics, and automation leads to smarter decision-making, improved efficiency, and better business outcomes.

5.5 Key Findings

The development and evaluation of the system led to several important findings that highlight its effectiveness and practical value.

Firstly, it was observed that **AI-based dynamic pricing is more efficient than traditional static pricing methods**. Static pricing does not adapt to changing conditions, whereas dynamic pricing continuously adjusts based on demand and external factors. This results in better revenue optimization and improved competitiveness.

Secondly, **external factors such as weather have a significant influence on food demand**. The analysis clearly shows that customer preferences change according to environmental conditions. By integrating weather data, the system is able to make smarter predictions and recommendations, which directly impact sales.

Another key finding is that **combining multiple data sources improves system performance**. The integration of demand data, weather data, and user behavior creates a more comprehensive model. This combination leads to higher prediction accuracy and better decision-making compared to using a single data source.

The system also demonstrates that **automation reduces dependency on manual processes**. Tasks such as price adjustment, demand prediction, and recommendation generation are performed automatically, saving time and reducing human errors. This increases operational efficiency and allows staff to focus on other important tasks.

Additionally, it was found that the system is **highly scalable and adaptable**. It can be extended to handle larger datasets, more users, and multiple restaurant branches. This makes it suitable for real-world deployment in both small and large-scale food businesses.

Another important finding is the **positive impact on customer engagement**. Personalized recommendations and dynamic pricing strategies create a more interactive and satisfying user experience. This increases customer retention and encourages repeat purchases.

Finally, the system highlights the importance of **data-driven decision-making** in modern business environments. By relying on accurate data and intelligent algorithms, businesses can make better strategic decisions, reduce risks, and maximize profitability.

Chapter 6: Conclusion and Future Scope

The AI-Driven Dynamic Food Menu and Pricing System has significant practical applications in real-world restaurant environments. By implementing this system, restaurant owners can make data-driven decisions instead of relying on guesswork. The system helps in identifying peak hours, popular food items, and customer preferences, enabling better planning of resources.

Moreover, dynamic pricing ensures that restaurants can maximize revenue during high demand while still attracting customers during low demand periods through reduced pricing. This flexibility allows businesses to stay competitive in a rapidly changing market. Additionally, personalized recommendations improve customer satisfaction, leading to increased customer retention and loyalty.

6.1 Business Benefits

The system provides multiple business advantages, including:

- **Increased Profitability:** By adjusting prices dynamically, restaurants can optimize their revenue streams.
- **Better Inventory Management:** Predicting demand helps in preparing the right quantity of food, reducing wastage.
- **Improved Customer Engagement:** Personalized suggestions enhance user experience.
- **Competitive Advantage:** Adoption of AI technology differentiates businesses from traditional competitors.
- **Operational Efficiency:** Automation reduces manual work and human errors.

These benefits make the system highly valuable for modern restaurants and food service providers.

6.2 Social and Economic Impact

The implementation of such intelligent systems can have a broader impact on society and the economy:

- **Reduced Food Wastage:** Efficient demand prediction leads to better utilization of food resources.
- **Affordable Pricing:** Customers can benefit from lower prices during off-peak hours.
- **Job Transformation:** While automation reduces manual tasks, it creates opportunities for skilled roles in AI and data analysis.
- **Sustainable Practices:** Optimized resource usage supports environmentally friendly operations.

6.3 Technical Contributions of the Project

This project contributes to the field of technology in several ways:

- Demonstrates the integration of **AI and Web Development** in a real-world application.
- Combines **multiple data sources** (user data, weather data, historical records) for better prediction.
- Implements a **full-stack solution (MERN Stack + Python ML model)**.
- Showcases how **APIs and machine learning models** can work together efficiently.

This makes the project a strong example of interdisciplinary application of computer science concepts.

6.4 Challenges Faced During Development

During the development of this project, several challenges were encountered:

- **Data Collection Issues:** Difficulty in obtaining large and clean datasets.
- **Model Accuracy:** Ensuring high accuracy in predictions required proper tuning and testing.
- **API Integration:** Handling real-time data from weather APIs required stable internet and proper error handling.
- **System Integration:** Connecting frontend, backend, and ML model smoothly was complex.
- **Performance Optimization:** Managing response time when handling large data sets was challenging.

Despite these challenges, appropriate solutions were implemented to ensure successful completion of the project.

6.5 Possible Real-World Deployment Strategy

For deploying this system in a real-world environment, the following steps can be considered:

1. **Cloud Deployment:** Use cloud platforms like AWS or Azure for scalability.
2. **Database Scaling:** Implement distributed databases for handling large data.
3. **Security Implementation:** Apply advanced encryption and authentication techniques.
4. **Continuous Monitoring:** Track system performance and update models regularly.
5. **User Training:** Train restaurant staff to use the system effectively.

This ensures smooth adoption and efficient functioning in real scenarios.

6.6 Comparison with Traditional Systems

Feature	Traditional System	Proposed AI System
Pricing	Fixed	Dynamic
Decision Making	Manual	Automated
Customer Experience	Basic	Personalized
Data Usage	Limited	Extensive
Efficiency	Low	High

Table 9.1: Comparison with traditional system

6.7 Ethical Considerations

While implementing AI-based systems, certain ethical factors must be considered:

- **Fair Pricing:** Prices should not exploit customers unfairly.
- **Data Privacy:** User data must be protected and used responsibly.
- **Transparency:** Customers should be aware of dynamic pricing policies.
- **Bias Reduction:** The system should avoid biased recommendations.

Addressing these concerns ensures responsible and ethical use of AI technology.

6.8 Long-Term Vision

The long-term vision of this project is to create a fully automated, intelligent restaurant management system that can:

- Predict customer behavior with high accuracy
- Automatically manage inventory and pricing
- Provide seamless customer interaction through AI assistants
- Integrate with smart devices and IoT systems

Such advancements can transform the food industry into a more efficient, data-driven ecosystem.

6.9 Final Remarks

In conclusion, the AI-Driven Dynamic Food Menu and Pricing System represents a significant step toward modernizing restaurant operations. By combining artificial intelligence, real-time data analysis, and automation, the system provides an innovative solution to traditional challenges.

The project not only improves business performance but also enhances customer satisfaction through personalized and dynamic services. Although there are certain limitations, the future scope of this system is vast and promising.

With further research, development, and real-world implementation, this system has the potential to revolutionize the food and hospitality industry by making it smarter, more efficient, and customer-centric.

References

Standard / Simple Format

- Pattern Recognition and Machine Learning – A comprehensive guide covering probabilistic approaches to machine learning, useful for understanding predictive modeling techniques.
- Machine Learning: A Probabilistic Perspective – Provides in-depth knowledge of machine learning concepts including classification, regression, and Bayesian methods.
- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow – Practical implementation of machine learning algorithms using modern tools and libraries.
- TensorFlow Documentation – https://www.tensorflow.org
  Used for building and training machine learning models for demand prediction.
- Scikit-learn Documentation – https://scikit-learn.org
  Provides simple and efficient tools for data mining and data analysis.
- Pandas Documentation – https://pandas.pydata.org
  Used for data preprocessing, cleaning, and manipulation.
- NumPy Documentation – https://numpy.org
  Supports numerical computations and array operations.
- Time Series Analysis – Important for forecasting demand trends based on historical data.
- Dynamic Pricing – A pricing strategy where prices are adjusted in real-time based on demand, supply, and external factors like weather.
- Demand Forecasting – Helps predict future customer demand using historical data and machine learning models.
- Research papers on:
  - "AI-based Dynamic Pricing Models in E-commerce"
  - "Demand Forecasting using Machine Learning Techniques"
  - "Weather-based Consumer Behavior Prediction"

Bibliography

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019).
   Database System Concepts (7th ed.). McGraw-Hill Education.
2. Pressman, R. S., & Maxim, B. R. (2015).
   Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill.
3. Kurose, J. F., & Ross, K. W. (2021).
   Computer Networking: A Top-Down Approach (8th ed.). Pearson.
4. Oracle Documentation.
   Retrieved from: https://docs.oracle.com
5. MDN Web Docs (Mozilla Developer Network).
   Retrieved from: https://developer.mozilla.org
6. W3Schools Online Web Tutorials.
   Retrieved from: https://www.w3schools.com
7. GeeksforGeeks.
   Retrieved from: https://www.geeksforgeeks.org
8. Official documentation of tools/technologies used in the project (e.g., Java, Node.js, HTML, CSS, etc.)

Appendices

Appendix A: Source Code

This section contains the complete or important parts of the source code used in the project

- Frontend code (HTML, CSS, JavaScript)
- Backend code (Node.js / Java / Python, etc.)
- Database queries (SQL / NoSQL)
- API integration code (if any)

Example structure:

- A.1 Frontend Code
- A.2 Backend Code
- A.3 Database Scripts
- A.4 Configuration Files

Appendix B: Extra Diagrams

This section includes additional diagrams that support the understanding of the system but are not included in the main chapters.

- System Architecture Diagram
- Data Flow Diagram (DFD) Level 0 / Level 1
- Use Case Diagram
- ER Diagram
- Class Diagram (if applicable)
- Sequence Diagram

Appendix C: Raw Results

This section contains the raw outputs and experimental/test results obtained during implementation and testing.

- Test case results (pass/fail table)
- Sample input and output screenshots
- Logs generated by the system
- Performance results (if any)
- Error logs and debugging outputs

Example table format:

Test Case ID	Input	Expected Output	Actual Output	Status
TC01	Login valid credentials	Dashboard load	Dashboard load	Pass
TC02	Invalid password	Error message	Error message	Pass

Table10.1: Table Format